Metro Navigator — High-Level Design (HLD)

Underground positioning with no satellites and no network

25 September 2026

Document control

Item	Value
JIRA	**NH-3779** — underground / metro indoor location (requirements and status: `04_NH-3779_Requirements_and_Status`)
Document	High-Level Design, version 2.1 (adds inertial speed, odometer, ride memory, tunnel radio learning, curve matching)
Product	Metro Navigator — Android app, module `:app` (uses library `:core`)
Date	25 September 2026
Status	Code complete; 82 automated tests passing (unit, component, physics-level ride simulations incl. a realistic line with a different speed per run, bundled data); lint 0 errors. **Pending: field validation on real rides (section 12).**
Companion documents	`02_Technical_Design.md` (algorithms, constants, tests) · `03_Technology_Guide.md` (plain-language guide)

1. Purpose and summary

Metro Navigator gives a metro passenger a continuous position — latitude/longitude, line, previous/next station, distance and time to the next station, stops to destination — **with no satellite signal and no mobile network**, as Google Maps does underground.

It follows the same four-part approach as Google's indoor/underground positioning:

1. **Radio anchors** — Wi-Fi fingerprints and Wi-Fi RTT ranging;
2. **Bluetooth beacon checkpoints**;
3. **Dead reckoning** — inertial navigation: the train's speed from integrated acceleration since the last stop (zero-velocity updates, accelerometer bias learnt at stops), blended with learnt run speeds; an inertial odometer that identifies each stop's station; and pedestrian dead reckoning (step counting) when walking;
4. **A Hidden Markov Model with map constraints** — all hypotheses lie on the metro line, plus physical constraints (a train that has left a station cannot be behind it).

On top, the phone **learns from every ride**, with no server: Wi-Fi/Bluetooth fingerprints at stations *and inside tunnels*, and the speed of every station-to-station run. The gyroscope's turns are checked against the line's curves as a safety net.

Two things differ from Google, by necessity, and the design compensates for both:

Google has	Apps cannot have	Metro Navigator instead
A crowd-sourced Wi-Fi / beacon database for the whole world	Access to that database offline	**Self-learnt** radio map on the phone (from GPS and trusted station stops); Google's database is still used **online** through Android's fused/network location
Transit data feeds	A guaranteed API while underground	**Automatic download of the whole current city** (any city in the world with metro data in OpenStreetMap) the first time the phone has a position and any network there; kept on the phone. A small bundled set (18 Indian cities, 336 KB) covers the one case a download cannot: first visit, never online, already underground

The strongest accuracy source underground is not radio but **station-stop detection**: every time the train stops at a platform the position is snapped exactly onto the station (0 m error) and the train's speed is recalibrated.

2. Scope

In scope - Positioning for every combination of GPS (good / weak / none) and network (good / weak / none). - Metro-specific navigation: line, stations, direction, destination, stops remaining, ETA, interchanges, express trains skipping stations, signal stops in tunnels. - Offline metro data (bundled + cached + optional download). - Wi-Fi fingerprinting, Bluetooth beacons, Wi-Fi RTT, IMU dead reckoning, step counting, HMM fusion, all on the phone. - Background operation with the screen off (foreground service + wake lock). - Daily log files and an on-screen history; warnings with one-tap fixes.

Out of scope - Map tiles / map UI (coordinates and station information are shown). - Server-side components: nothing is uploaded; the learnt radio map stays on the phone. - Road-vehicle navigation (a generic vehicle model exists only as a fallback).

3. Requirements

3.1 Functional

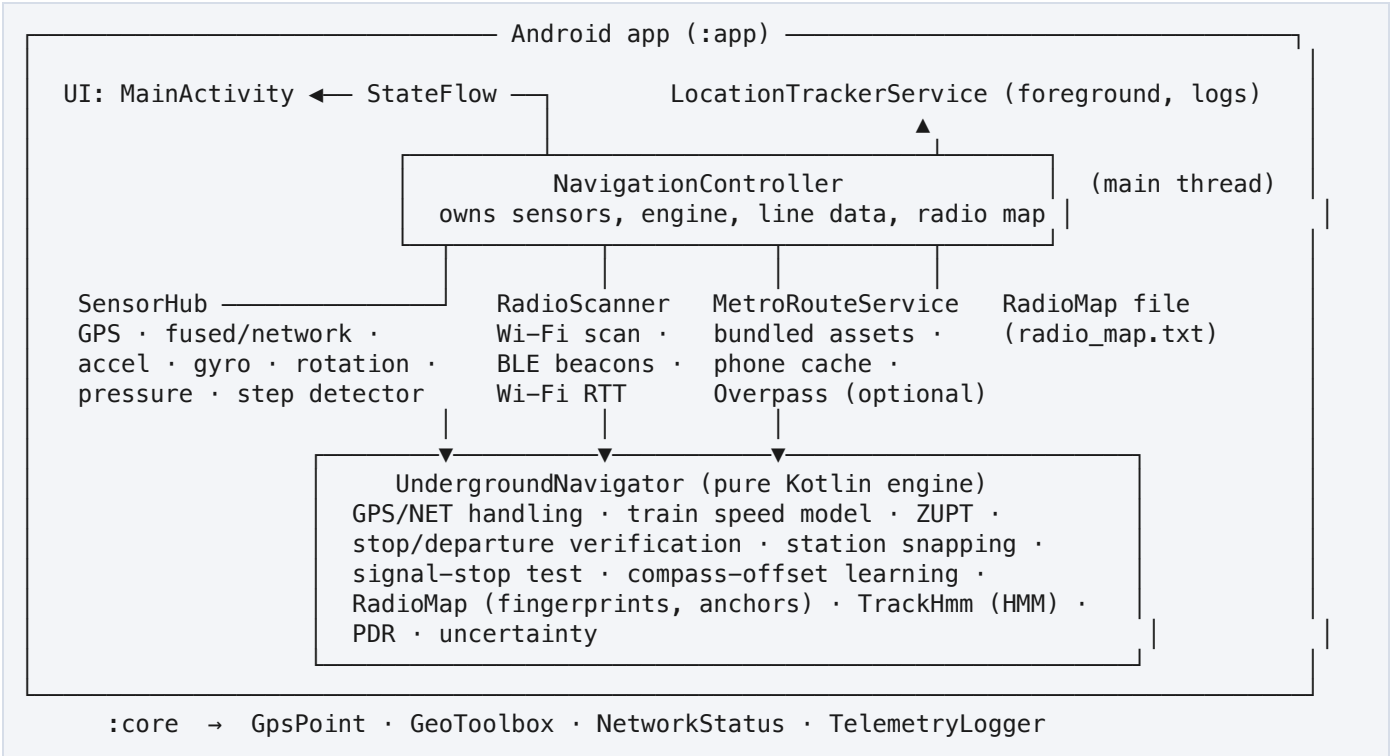
ID	Requirement
F1	Show the current position at all times once a first position is known; mark its source (GPS / NET / RADIO / TRACK / DR) and its ± error.
F2	Work with no network: all line data, radio map and computation on the phone.
F3	Work even if the metro API is unavailable: bundled line data for supported cities.
F4	Recognise every station stop underground and show previous / next station, distance, ETA.
F5	Determine direction of travel without GPS (departure acceleration) and pick the line at interchanges.
F6	Estimate speed without GPS; show zero while stopped.
F7	Distinguish a real stop from a smooth glide, and a departure from phone handling.
F8	Do not mistake a signal stop in the tunnel for a station; handle express trains.
F9	Learn Wi-Fi / Bluetooth fingerprints automatically and use them to correct the position.

ID	Requirement
F10	Count steps and move the position when walking with no GPS.
F11	Log every position and event to a daily file.

3.2 Non-functional

ID	Requirement	How it is met
N1	Position update ≥ 1 Hz underground	10 Hz engine, 4 Hz UI, 0.5 Hz log
N2	Error at stations ≈ 0 m	stop snapping (verified in all simulations)
N3	Error between stations ≤ few hundred metres worst case	100–370 m worst case in simulation (section 11)
N4	No UI freeze	engine O(line length) per tick; network/file I/O off the main thread
N5	Survive bad data	every input validated; outlier gates; atomic file writes; corrupt files skipped
N6	Battery	GPS 1 Hz, Wi-Fi scan 30 s, BLE low-latency only while navigating; stops with the app
N7	Privacy	no upload; radio map is local; OSM attribution kept

4. Architecture



Key design choices

1. **One pure-Kotlin engine** (`UndergroundNavigator` + `TrackHmm` + `RadioMap` + `Pdr`) with no Android types, so every behaviour can be simulated and unit-tested on a PC.

2. **1-D track state** underground (distance along the line + direction), converted to lat/lon only for output. This is the map constraint and removes most of the error.
3. **Every input is optional.** Missing GPS, network, Wi-Fi, Bluetooth, RTT, step detector, orientation sensor or line data reduces accuracy but never stops the app.
4. **Learning only from trusted positions** (good GPS, trusted station stops), never from its own estimates, so errors cannot feed on themselves.

5. Data sources and data flow

Source	Rate	Used for
GNSS (LocationManager GPS provider)	1 Hz	truth above ground; learning speed, compass offset, radio map
Fused / network location	2 s	absolute corrections without GPS (Google's Wi-Fi/cell positioning when online)
Linear acceleration + rotation vector	50 Hz → 10 Hz windows	speed, stop/departure checks, direction, compass offset
Gyroscope	50 Hz	stillness detection
Barometer	~5 Hz	stairs/escalator detection (going underground)
Step detector	per step	walking detection, PDR
Wi-Fi scans	30 s (+ other apps' scans)	fingerprints
BLE beacon scan	continuous, reported every 2 s	checkpoints, fingerprints
Wi-Fi RTT	2 s when responders are known	ranging
Metro lines	once per install/update (bundled), on demand (download)	map constraint, stations

Flow per tick (10 Hz): sensor window → `MotionSample` → engine updates speed, stop / departure logic, position along the line, uncertainty → `NavState` → UI / notification; every 2 s without GPS a `PositionEvent` is logged.

Flow per radio scan: `RadioScan` → engine: *learn* (if the position is trusted) or *localise* (HMM forward step on the line / radio fix off the line).

6. Positioning behaviour by situation

Situation	Behaviour
GPS good	Position = GPS (jump-filtered). Learn cruise speed, compass offset, radio fingerprints. Keep the line / direction / distance along it ready.
GPS lost on a line	Carry position over the detection gap; continue along the track with the speed model; hold at the next station until the train stops there.
Train stops	Verify braking (integrated velocity back to ~0); decide station vs. signal stop; snap to station (0 m); learn speed; learn radio at the platform.

Situation	Behaviour
Train departs	Wait for a real, one-directional push (not hand jiggle); set / confirm direction; learn compass offset; choose line at interchanges.
Radio heard (known)	HMM forward step on the line; correct position if the belief is unambiguous; recognise stations passed without stopping; confirm stops.
Network fix	Gated 1-D Kalman update along the line (or 2-D blend off the line).
Walking, no line	PDR: steps × step length along the phone's heading.
Riding, no line data	Vehicle model along the last heading with ZUPT and departure checks.
Cold start underground	Radio map gives the first position (if learnt) → lock onto the line → departure gives direction.
GPS returns	Confirm with two consistent fixes; log the drift of the estimate.

7. Metro data for the current city, anywhere in the world

Navigation underground never waits for, or depends on, a server. Line data reaches the phone in advance:

1. **Automatic, current city, worldwide (primary).** The first time the phone has a position (GPS or network) **and** any network in an area — at home, on the street, at the station entrance — the app downloads the **whole metro network within 30 km** from OpenStreetMap and stores it. If the server cannot deliver a whole city, it immediately saves the lines within 2 km and retries the city later with back-off, and again whenever a network appears. The area is remembered for 7 days, then silently refreshed when online. Works in any city whose metro is mapped in OpenStreetMap.
2. **On the phone (cache).** Everything downloaded stays usable offline, indefinitely.
3. **Bundled safety net.** `scripts/build_metro_bundle.py` (build time) packs chosen cities into the APK — currently 18 Indian cities, 336 KB — for the one case the automatic download cannot cover: first visit to a city, never online there, already underground. Any city worldwide can be added to the script.
4. **No line at all:** the engine still works with the vehicle model, PDR, radio and network (lower accuracy, section 11).

8. Radio positioning (Wi-Fi, BLE, RTT) and the HMM

- **Learning:** scans taken with GPS ≤ 20 m accuracy (fix ≤ 3 s old) or at a station reached by a trusted stop (error before the snap ≤ 300 m, radio not contradicting it) are stored as fingerprints (position, σ, mean RSSI per transmitter, merged within 20 m). Each transmitter gets an anchor at its strongest sighting. Capacity 5000 places, 30 transmitters each.
- **Matching:** RSSI distance over common transmitters (σ = 8 dB), weighted by coverage; a single Wi-Fi AP is not enough, a single beacon is.
- **HMM:** hidden state = 10 m cell of the current line; transition = the engine's own motion (shift + blur by its uncertainty growth, 2 % "anywhere"); emission = mixture of fingerprint Gaussians × beacon proximity × RTT ranges, with a floor so no single reading is decisive; output = dominant mode. Applied only if the mode holds ≥ 60 % of the probability.
- **Safety rules:** never move the train backwards by a small radio correction; never hold the position at a station the radio saw the train pass; never learn from the engine's own estimates.

9. Speed, stops and departures without GPS

9.1 Speed — inertial navigation with zero-velocity updates

At every accepted stop the train's velocity is exactly zero. From there the app integrates the world-frame acceleration (bias removed) into a velocity vector; its size is the measured speed. The accelerometer bias is learnt while the train stands still (30 s averaging, kept across stops). The measured speed is blended with the model speed (the run's learnt cruise, or the line's) by their errors: the measurement's error grows with time since the stop (0.5 m/s + 0.1 m/s per second), the model's is 12 % of the speed on a learnt run and 30 % otherwise. The position error then grows with that fused speed error. Simulated speed error: ~1 m/s.

9.2 Stops, stations and signal stops

Question	Evidence	Fallback
Is the phone still?	accelerometer spread + gyroscope + step detector (SensorHub)	—
Is the train really stopped (not gliding)?	integrated velocity since the last stop back near 0 (≤ max(3 m/s, 35 % of its peak, 0.04 m/s per second of run))	still ≥ 15 s is accepted
Which station?	**odometer**: drift-corrected inertial distance since the last trusted station → the station at that distance (±max(80 m, 10 %)); none → signal stop	probability of station vs. uniform signal stop (85/15) using the engine's error
Has the train really left?	≥ 1.5 m/s of velocity change in 6 s with ≥ 50 % directional consistency	20 s of motion-type vibration
Which way?	departure push vs. line bearing (compass-offset corrected); a known direction flips only on a clearly opposite push	user can tap *Flip* or pick a destination

9.3 Learning from every ride (no backend)

- **Ride memory:** each run's moving time → its cruise speed (trapezoid with 1 m/s² acceleration/braking); used as the speed target the next time.
- **Tunnel radio:** scans heard during a run between two trusted stops are placed along the tunnel with the drift-corrected inertial distance and stored as fingerprints.
- **Curve matching:** gyroscope turn since the last station vs. the line's turn to each point ahead; an estimate that contradicts the measured turn moves to the nearest agreeing point.

10. Non-functional design

- **Threading:** all engine state on the main thread; downloads, parsing and file I/O on `Dispatchers.IO`; radio map serialised on the main thread (cheap string building) and written on IO (temp file + rename).
- **Lifecycle:** sensors and radio run while the screen or the logging service holds the controller; released when neither does; radio map saved on release and every 60 s if changed.
- **Power:** partial wake lock only while the logging service runs (12 h cap); BLE scan only while navigating; Wi-Fi scans at the Android foreground limit.

- **Permissions:** location (fine), activity recognition, notifications, nearby devices (Bluetooth scan, Wi-Fi RTT); every one optional except location. The UI explains each missing one with a one-tap fix.
- **Privacy:** radio map and logs stay on the phone.

11. Verified accuracy (simulation, no satellites and no network underground)

Physics-level simulation: curved 20 km line (two 45° curves), 17 stations, accelerate 1.0 m/s², brake 1.1 m/s², 25 s dwell; accelerometer with tangential and centripetal acceleration, 25–60° compass error, bias and noise; gyroscope with noise and bias; radio path loss. "Realistic line": each run has its own cruise speed (11–25 m/s), the same on every ride.

Scenario	Stations	Stop error	Moving error mean / max	Speed RMS
20 km tunnel, 80 km/h	16/16	0 m	16 / 116 m	1.1 m/s
Realistic line, 6 seeds	all	0 m	12–17 / 67–164 m	0.7–0.9 m/s
43 km/h · 108 km/h	16/16	0 m	29 / 158 m · 17 / 257 m	1.0 · 1.7 m/s
False "still" readings while cruising	16/16	0 m	16 / 115 m	1.1 m/s
Phone jiggled / turned at platforms	16/16	0 m	14–17 / 82–144 m	~1 m/s
Signal stop in tunnel	16/16	0 m	17 / 91 m	1.2 m/s
Boarding underground, reverse direction	8/8	0 m	30 / 280 m	2.0 m/s
Express (6 skipped), nothing learnt	all	0 m	67–91 / 620–800 m	~1.1 m/s
Express after one learning ride	all	0 m	22–30 / 113–152 m	1.1 m/s
Second ride, everything learnt	all	0 m	18–22 / 117–125 m	~1 m/s
Cold start underground, radio map only	6/6	0 m	19 / 121 m	2.4 m/s
No orientation sensor, realistic line	all	0 m	57–61 / 425–458 m	2.8 m/s
No line data (6 km)	—	454 m	228 / 454 m	1.2 m/s

Feature value measured on identical rides (same random seeds): - inertial speed: every station right on the realistic line (before: several rides off by a station), average error roughly halved, speed error 3–4 → ~1 m/s; - odometer: express and signal stops always identified correctly; - learnt radio + run memory: express rides' worst case 620–800 m → 113–152 m; - curve matching: no change when the estimate is consistent (the normal case); removes >300 m when the speed estimate is badly wrong.

12. Field validation (required before production)

1. Install on 2–3 phone models (one without RTT, one without barometer).
2. Ride each line end-to-end twice (first ride teaches the radio map): enable logging.
3. After each ride compare the log with the station timetable and the GPS-recovery drift events; check: all stops recognised, no wrong stops, direction events correct.
4. Tune only through the constants listed in the Technical Design (section 9), re-run the simulations, repeat. Acceptance: 100 % stations correct, stop error 0 m, 95 % of moving errors ≤ 300 m.

13. Risks and mitigations

Risk	Mitigation
Phone orientation sensor missing	inertial speed, odometer and stop checks disabled automatically; simpler model with wider error margins keeps every station in simulation
Accelerometer bias / tilt drift	bias learnt at every stop; drift removed between stops (ZUPT); allowances grow with run time
Magnetometer disturbed by train	compass offset learnt; speed, stops and curve matching are heading-independent
Wrong station snap poisons learning	learning only after trusted snaps (odometer / radio / small error); radio conflict blocks learning
Radio looks the same before and after a station	kinematic constraint (not behind the station just left); tempered evidence; act only on clear gains
Express train on a never-ridden line	stops still right (odometer); display lag while passing — removed after one ride
Android Wi-Fi scan throttling	passive results from other apps; BLE continuous; fingerprints optional
Bundled/cached data outdated	automatic city refresh when online; re-run the bundle script each release
Simulation ≠ reality	field validation (section 12) before production

14. Glossary

See the Technology Guide, section 7.